

Operations Nerd

A config-first, multi-vertical operations engine for small and mid-size businesses

Research proposal prepared for Professor Dennis Sasha, NYU

1. Problem Statement

Most operational software is built narrowly. A tool that tracks inventory does not do accounting, and a tool that does accounting does not do outreach. Businesses end up assembling a stack of point tools rather than relying on one system that generalizes across the operational work they actually need done.

Underneath the specific tasks, most small and mid size businesses, whether a hospital, a clinic, or a real estate office, are doing the same handful of things:

- Relationship management: keeping track of contacts, patients, clients or leads
- Sales and payment tracking: who owes what, who paid, what was sold
- Outreach tracking: reach outs, follow ups, and their outcomes
- Accounting: the financial record of the above

The research question is whether a single generalized system can handle these four functions well enough across different verticals to replace the assortment of special purpose tools businesses use today, and where the limits of that generalization are.

1.1 Relationship to CustomNerd

This project follows the same architectural thesis as CustomNerd (Kashyap, Wu, Shasha et al.): one configuration-first pipeline can serve many domains if domain-specific behavior is externalized into saved-state configuration rather than code. CustomNerd applied this to expertise-driven question answering and was evaluated as a research contribution against RAG and agentic baselines. Operations Nerd applies the same underlying pattern to business operations tracking, is built independently from scratch with no shared code, and is scoped as an application-based deliverable meant for real industry use rather than an academic benchmark study.

2. State of the Art

2.1 Industry direction

The commercial market has moved toward vertical AI agents rather than generalist ones, because a vertical agent's value comes from its data, tool integrations, and codified workflow knowledge, not the underlying model alone. Enterprise vertical AI spend roughly tripled in 2025, led by healthcare and legal, and a large share of the broader vertical SaaS market is projected to be reshaped by AI agents over the next few years.

2.2 Academic direction

Benchmarks such as CRMArena and CRMArena Pro test how well LLM based agents handle realistic CRM tasks across varied scenarios, giving a way to measure generalization loss rather than assume it. Separate work on business process discovery through agentic generative AI, and on merging specialist agents into a single generalist model, is actively studying the same generalist versus specialist tradeoff this project is built around.

2.3 The gap

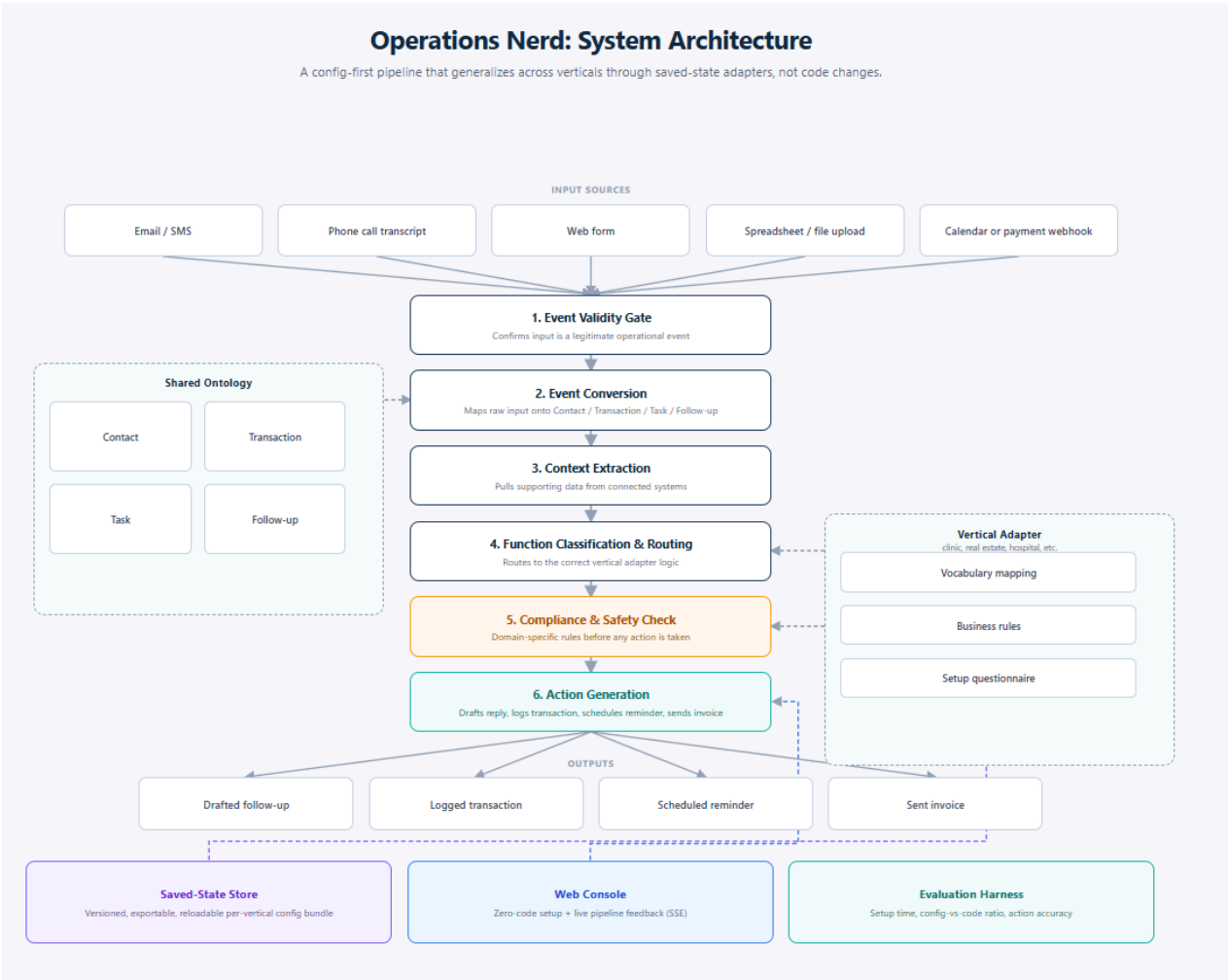
No product or published system cleanly solves one system, many verticals, minimal setup for all four functions together. The table below summarizes the closest existing categories and why each falls short.

Category	Closest examples	Why it falls short
Horizontal no-code builders	Zapier Central, Relevance AI, Lindy, Airtable Omni, n8n	Manual setup per business, no reusable saved-state snapshot
Vertical point solutions	Salesforce, HubSpot, monday CRM, ServiceAgent.ai	Accounting is always a bolt-on integration, never native
AI operating system platforms	PwC agent OS, Ema, Decidr.ai	Enterprise-scale and consulting-led, not a fast self-serve config tool for a solo clinic or office

Small businesses currently run a median of five separate AI tools rather than one consolidated system, which is itself evidence for the problem statement: if a generalized system worked well enough, it would be replacing several point tools at once.

3. Proposed Architecture

The design keeps a single shared core generalized across verticals, with a thin, swappable adapter layer carrying the domain-specific knowledge. This directly targets the generalist versus specialist tradeoff surfaced in the academic literature above.



3.1 Shared ontology

Every vertical adapter maps onto four universal primitives: Contact, Transaction, Task, and Follow-up. A patient is a Contact, an invoice is a Transaction, a recall reminder is a Follow-up. This is what makes the four functions generalizable across verticals rather than each vertical needing its own data model.

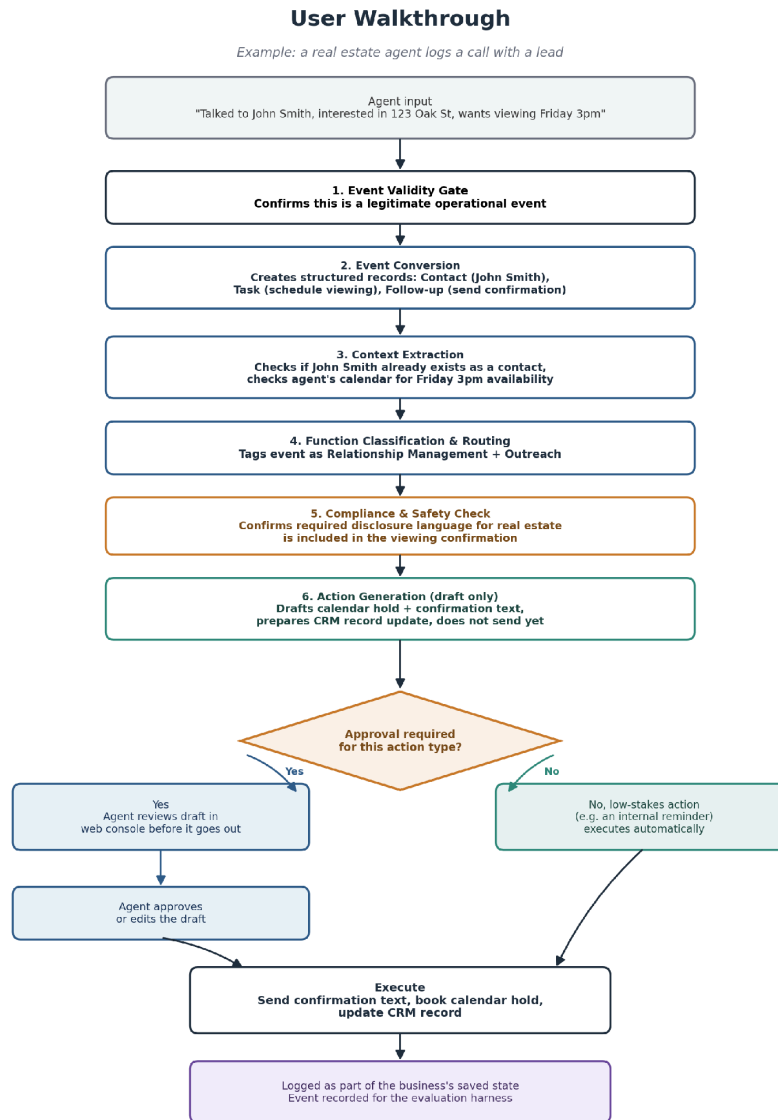
3.2 Nine-stage pipeline

#	Stage	What it does
1	Event Validity Gate	Confirms an incoming input (email, call transcript, form, spreadsheet row) is a legitimate operational event before anything downstream runs.
2	Event Conversion	Converts the raw input into a structured record against the shared ontology: Contact, Transaction, Task, or Follow-up.
3	Context Extraction	Pulls supporting data from connected systems (calendar, payment processor, existing contact record) needed to act on the event.
4	Function Classification & Routing	Determines which of the four functions the event belongs to and routes it to the matching vertical adapter logic.
5	Compliance & Safety Check	Domain-specific checks before any action is taken, for example HIPAA-relevant handling in a clinic or licensing disclosure language in real estate.
6	Action Generation	Produces the actual output: a drafted reply, a logged transaction, an invoice line, or a scheduled reminder.
7	Adapter Setup Questionnaire	A short, one-time setup flow per business that configures the vertical adapter instead of personalizing a single query.
8	Saved-State Export	Bundles the vertical adapter, ontology mappings, integration credentials, and business rules into one versioned, reloadable snapshot.
9	Live Pipeline Feedback	Streams stage-by-stage progress to the business owner in the web console as each event is processed.

Stage 6, Action Generation, is the main departure from the CustomNerd pattern and the primary new engineering surface: it produces a real-world side effect, sending a message, creating a calendar entry, posting a ledger line, rather than only generating text. This also raises the stakes on stage 5, since a wrong invoice or a missed compliance check is a materially worse failure than a wrong answer.

4. User Walkthrough

To make the pipeline concrete, here is what happens end to end for a single event, illustrated with a real estate example: an agent logging a call with a lead.



Two design decisions surface directly from tracing this walkthrough. First, action generation in stage 6 produces a draft, not an executed action, execution is a separate step gated by an approval check. Second, whether that approval is required is itself configurable per action type: a low-stakes internal reminder can execute automatically, while anything customer-facing or financial, a sent message, a booked appointment, an invoice, should default to requiring human approval until the compliance and safety stage has enough track record to be trusted unsupervised. This is a deliberate, named design decision rather than an implementation detail, since it directly affects how much a business has to trust the system on day one.